Scripts del guion del video - TGS

Los 6 scripts que se usan en el video de entrega, en orden de uso:

#	Script	Que hace
1	estado.sh	Estado del sistema (contenedores, sitio, backups, ultima caida)
2	respaldar.sh	Genera un backup de MySQL en backups/tgs_*.sql
3	caida.sh	Simula la caida: baja los contenedores
4	instalar.sh	Reinstala: genera .env/.env.local, levanta Docker y sincroniza la contrasena
5	restaurar.sh	Restaura el backup elegido en MySQL
-	_entorno.sh	Libreria de helpers que importan los anteriores (getent, permisos, mensajes)

1. scripts/estado.sh

estado.sh

```
#!/usr/bin/env bash
# ================================================================
# TGS - Estado del sistema
#
# Muestra si los contenedores estan arriba, si la base responde y si
# el sitio web contestan. Es el comando para comprobar, antes y despues
# de cada paso del video, que el servidor esta en pie.
# ================================================================
set -euo pipefail

source "$(dirname "${BASH_SOURCE[0]}")/_entorno.sh"

cd "$TGS_RAIZ"

tgs_cargar_entorno

echo "============================================================"
echo " TGS - Estado del sistema"
echo "============================================================"
echo

echo "  Contenedores"
echo "  -------------"
docker compose ps
echo

# Estado de la base
if docker compose ps --status running --services 2>/dev/null | grep -q '^db$'; then
    if docker compose exec -T db mysqladmin ping -h 127.0.0.1 -uroot -p"$DB_ROOT_PASS" --silent >/dev/null 2>&1; then
        echo "  Base de datos: ACTIVA  ($DB_NAME en el contenedor tgs_db)"

        USUARIOS="$(docker compose exec -T db mysql -uroot -p"$DB_ROOT_PASS" -N -B -e \
            "SELECT COUNT(*) FROM \`$DB_NAME\`.\`usuario\`;" 2>/dev/null | tr -d '[:space:]' || echo '?')"
        RESERVAS="$(docker compose exec -T db mysql -uroot -p"$DB_ROOT_PASS" -N -B -e \
            "SELECT COUNT(*) FROM \`$DB_NAME\`.\`reserva\`;" 2>/dev/null | tr -d '[:space:]' || echo '?')"
        echo "    usuarios registrados : ${USUARIOS:-?}"
        echo "    reservas registradas : ${RESERVAS:-?}"
    else
        echo "  Base de datos: CONTAINER ARRIBA pero MySQL no responde"
    fi
else
    echo "  Base de datos: CAIDA (el contenedor tgs_db no esta corriendo)"
fi
echo

# Estado del sitio web
if docker compose ps --status running --services 2>/dev/null | grep -q '^app$'; then
    CODIGO="$(curl -s -o /dev/null -w '%{http_code}' "http://localhost:${APP_PUERTO}/" 2>/dev/null || echo '000')"

    if [ "$CODIGO" = "200" ]; then
        echo "  Sitio web: RESPONDIENDO  http://localhost:${APP_PUERTO}  (HTTP $CODIGO)"
    else
        echo "  Sitio web: CONTENEDOR ARRIBA pero la pagina devuelve HTTP $CODIGO"
    fi
else
    echo "  Sitio web: CAIDO (el contenedor tgs_app no esta corriendo)"
fi
echo
```

```
# Ultimo respaldo disponible
if [ -d "$TGS_RAIZ/backups" ]; then
    ULTIMO="$(find "$TGS_RAIZ/backups" -maxdepth 1 -name 'tgs_*.sql' -printf '%T@ %p\n' 2>/dev/null \
        | sort -rn | head -1 | cut -d' ' -f2- || true)"

    if [ -n "$ULTIMO" ]; then
        echo "  Ultimo respaldo: $(basename "$ULTIMO")  ($(du -h "$ULTIMO" | cut -f1))"
    else
        echo "  Ultimo respaldo: ninguno todavia (bash scripts/respaldar.sh)"
    fi
else
    echo "  Ultimo respaldo: la carpeta backups/ todavia no existe"
fi
echo
```

2. scripts/respaldar.sh

respaldar.sh

```
#!/usr/bin/env bash
# ============================================================================
# TGS - Respaldo de la base de datos
#
# Saca un mysqldump del CONTENEDOR de MySQL y lo guarda en
# backups/tgs_<fecha>.sql. Gira los respaldos viejos para no acumular.
# ============================================================================
set -euo pipefail

source "$(dirname "${BASH_SOURCE[0]}")/_entorno.sh"

cd "$TGS_RAIZ"

echo "============================================================"
echo " TGS - Respaldo de la base de datos"
echo "============================================================"

tgs_cargar_entorno

# Sin este chequeo el mysqldump fallaria con un error poco claro
if ! docker compose ps --status running --services 2>/dev/null | grep -q '^db$'; then
    echo "ERROR: el contenedor de la base de datos no esta corriendo." >&2
    echo "       Levantalo con: bash scripts/instalar.sh" >&2
    exit 1
fi

mkdir -p "$TGS_RAIZ/backups"

# mysqldump dentro del contenedor, volcado a un archivo en el host
NOMBRE="tgs_$(date +%Y%m%d_%H%M%S).sql"
DESTINO="$TGS_RAIZ/backups/$NOMBRE"

echo
echo "  Base de datos : $DB_NAME"
echo "  Generando     : backups/$NOMBRE"
echo

docker compose exec -T db mysqldump \
    -u root \
    -p"$DB_ROOT_PASS" \
    --single-transaction \
    --routines \
    --triggers \
    --add-drop-table \
    --default-character-set=utf8mb4 \
    "$DB_NAME" > "$DESTINO"

# Un respaldo vacio no sirve para nada: se descarta
if [ ! -s "$DESTINO" ]; then
    rm -f "$DESTINO"
    echo "ERROR: el respaldo salio vacio, se elimino." >&2
    exit 1
fi

chmod 600 "$DESTINO"

TAMANO="$(du -h "$DESTINO" | cut -f1)"
TABLAS="$(grep -c '^CREATE TABLE' "$DESTINO" || true)"

echo "[OK]   Respaldo generado: backups/$NOMBRE"
echo "       Tamano: $TAMANO   Tablas: $TABLAS"
echo

# Rotacion: conserva solo los RESPALDO_CONSERVAR respaldos mas recientes
CONSERVAR="${RESPALDO_CONSERVAR:-5}"
TOTAL="$(find "$TGS_RAIZ/backups" -maxdepth 1 -name 'tgs_*.sql' | wc -l)"
```

```
if [ "$TOTAL" -gt "$CONSERVAR" ]; then
    echo "       Rotando respaldos (conservando los $CONSERVAR mas recientes):"
    find "$TGS_RAIZ/backups" -maxdepth 1 -name 'tgs_*.sql' -printf '%T@ %p\n' \
        | sort -rn \
        | tail -n "+$((CONSERVAR + 1))" \
        | cut -d' ' -f2- \
        | while read -r viejo; do
            rm -f "$viejo"
            echo "         eliminado $(basename "$viejo")"
        done
fi

echo
echo "  Para restaurar este respaldo:"
echo "    bash scripts/restaurar.sh $NOMBRE"
echo
```

3. scripts/caida.sh

caida.sh

```
#!/usr/bin/env bash
# ======================================================================
# TGS - Simulacion de la caida del servidor
#
# Uso:
#   bash scripts/caida.sh          borra contenedores Y volumen (perdida total)
#   bash scripts/caida.sh --suave  solo detiene, conserva los datos
#
# La opcion por defecto usa "down -v", que es la caida que hay que
# mostrar en el video: se lleva por delante el volumen de MySQL y con
# el los datos. Por eso hay que respaldar antes de ejecutarla.
# ======================================================================
set -euo pipefail

source "$(dirname "${BASH_SOURCE[0]}")/_entorno.sh"

cd "$TGS_RAIZ"

MODO="${1:-}"

echo "============================================================"
echo " TGS - Simulacion de caida del servidor"
echo "============================================================"

tgs_cargar_entorno

if [ "$MODO" = "--suave" ]; then
    echo
    echo "  Modo: detencion suave (los datos se conservan)"
    echo
    docker compose stop
    echo
    echo "[OK]   Contenedores detenidos. Los datos siguen en el volumen."
    echo "       Para volver:  bash scripts/instalar.sh"
    echo
    exit 0
fi

# Aviso claro: esta operacion destruye los datos del contenedor
RESPUESTA=""
if [ -t 0 ]; then
    read -r -p "  Esto BORRA el volumen y todos los datos. Continuar? (si/NO): " RESPUESTA
else
    RESPUESTA="si"
fi

if [ "$RESPUESTA" != "si" ]; then
    echo "  Cancelado."
    exit 0
fi

echo
echo "  Deteniendo y eliminando contenedores y volumen..."
echo

# -v es lo que elimina el volumen db_data: aqui se pierden los datos
docker compose down -v

echo
echo "============================================================"
echo " CAIDA SIMULADA"
echo "============================================================"
echo
echo "  Estado actual:"
```

```
docker compose ps 2>/dev/null || true
echo
echo "  El sitio http://localhost:${APP_PUERTO} ya no responde."
echo
echo "  Para recuperar el sistema:"
echo "    1) bash scripts/instalar.sh    (crea variables y levanta de nuevo)"
echo "    2) bash scripts/restaurar.sh   (recupera la base desde el respaldo)"
echo
```

4. scripts/instalar.sh

instalar.sh

```
#!/usr/bin/env bash
# ======================================================================
# TGS - Instalacion completa del proyecto
#
# Que hace, en orden:
#   1. Verifica que Docker este disponible
#   2. Crea las variables LOCALES (.env.local) a partir de la plantilla
#   3. Muestra la configuracion final mezclando global + local
#   4. Construye la imagen y levanta los contenedores
#   5. Espera a que MySQL este listo
#   6. Se asegura de que existan el esquema y los usuarios
#
# Es idempotente: se puede volver a ejecutar sin romper nada.
# ======================================================================
set -euo pipefail

source "$(dirname "${BASH_SOURCE[0]}")/_entorno.sh"

cd "$TGS_RAIZ"

echo "============================================================"
echo " TGS - Instalacion del proyecto"
echo "============================================================"
echo

echo "[1/6] Verificando Docker"
tgs_verificar_docker
echo

echo "[2/6] Variables de configuracion"

# MySQL graba las claves en el volumen cuando se crea, y no las vuelve a
# leer. Si el volumen ya existe pero .env.local no, generar claves nuevas
# deja la base inaccesible, asi que se avisa ANTES de tocar nada.
if [ -f "$TGS_RAIZ/.env" ] && [ ! -f "$TGS_RAIZ/.env.local" ] \
   && docker volume ls -q | grep -q .; then
    if docker compose ps -q db 2>/dev/null | grep -q .; then
        cat >&2 <<'AVISO'

[AVISO] Hay un volumen de datos de una instalacion anterior, pero no
        existe .env.local. Se van a generar claves nuevas que MySQL no
        va a reconocer, porque las del volumen quedaron fijadas al
        crearse.

        Si querias conservar los datos, cancela con Ctrl+C y restituye
        el .env.local anterior. Si querias empezar de cero, segui y
        despues borra el volumen con:  bash scripts/caida.sh

AVISO
    fi
fi

# El .env global no se versiona: en un clone nuevo no existe y hay que
# generarlo antes de poder leer nada.
tgs_crear_env_global
tgs_cargar_entorno
tgs_crear_env_local

# Aunque .env.local ya existiera, sus permisos pueden haber quedado en 600
# de una corrida anterior y Apache no podria leerlo dentro del contenedor.
tgs_ajustar_permisos_env_local

# Hay que volver a cargar: recien creados, .env y .env.local ya traen
# los valores definitivos de esta maquina.
tgs_cargar_entorno

echo "       .env        (global)  -> presente"
echo "       .env.local  (local, prioritaria)  -> $TGS_RAIZ/.env.local"
echo
echo "       Valores que queden aplicados:"
printf "         DB_HOST      = %s\n" "$DB_HOST"
printf "         DB_NAME      = %s\n" "$DB_NAME"
printf "         DB_USER      = %s\n" "$DB_USER"
```

```
printf "         APP_PUERTO   = %s\n" "$APP_PUERTO"
printf "         APP_ENV      = %s\n" "$APP_ENV"
echo "         DB_PASS      = ********  (oculta a proposito)"
echo

echo "[3/6] Construyendo la imagen de la aplicacion"
docker compose build
echo

echo "[4/6] Levantando los contenedores"
docker compose up -d
echo

echo "[5/6] Esperando a la base de datos"
tgs_esperar_db 40
echo

echo "[6/6] Comprobando esquema y usuarios"

# MySQL graba MYSQL_USER / MYSQL_PASSWORD / MYSQL_ROOT_PASSWORD en el
# volumen solo la primera vez que se crea. Despues, cambiar .env.local no
# cambia nada dentro de MySQL: el volumen sigue con la clave vieja.
#
# Eso deja dos casos distintos:
#   a) el usuario tgs no entra, pero root si -> se puede arreglar
#   b) root tampoco entra -> el volumen tiene una clave que no conocemos,
#      y sin ella los datos son inalcanzables
# El caso (b) no se puede recuperar solo: hay que avisar y parar.
tgs_puede_entrar() {
    docker compose exec -T db mysql -u"$1" -p"$2" -e "SELECT 1;" >/dev/null 2>&1
}

if tgs_puede_entrar "$DB_USER" "$DB_PASS"; then
    echo "[OK]   las credenciales de .env.local coinciden con el volumen"
elif tgs_puede_entrar root "$DB_ROOT_PASS"; then
    echo "[AVISO] el usuario '$DB_USER' no acepta la password de .env.local"
    echo "        (el volumen ya existia con otra clave). Sincronizando..."
    docker compose exec -T db mysql -uroot -p"$DB_ROOT_PASS" -e \
        "ALTER USER '$DB_USER'@'%' IDENTIFIED BY '$DB_PASS'; FLUSH PRIVILEGES;" >/dev/null 2>&1
    echo "[OK]   password del usuario '$DB_USER' sincronizada"
else
    cat >&2 <<'AVISO'

[ERROR] El volumen de datos ya existe pero sus credenciales no coinciden
        con .env.local, y no se puede entrar ni como root.

        MySQL guarda las claves DENTRO del volumen, y solo las lee la
        primera vez que se crea. Por eso regenerar .env.local con el
        volumen ya existente deja la base inaccesible: las claves nuevas
        no coinciden con las que quedaron guardadas.

    Tenes dos salidas, segun lo que necesites:

      1. Queria empezar de cero (no me importan los datos)
         bash scripts/caida.sh      # borra el volumen
         bash scripts/instalar.sh   # y reinstalla con claves nuevas

      2. Queria conservar los datos
         Volve a poner el .env.local que tenias antes. Si lo perdiste,
         no hay forma de recuperarlo: solo queda restaurar un respaldo
         sobre un volumen limpio, como en la opcion 1.

AVISO
    exit 1
fi

echo

# En un volumen recien creado MySQL ya ejecuto schema.sql y seed.sql.
# Si el volumen ya existia sin tablas, se aplican aqui a mano.
if docker compose exec -T db mysql -uroot -p"$DB_ROOT_PASS" \
       -e "USE \`$DB_NAME\`; SELECT 1 FROM usuario LIMIT 1;" >/dev/null 2>&1; then
    echo "[OK]   el esquema y los usuarios ya estaban cargados"
else
    echo "[....] aplicando db/schema.sql"
    docker compose exec -T db mysql -uroot -p"$DB_ROOT_PASS" -e "CREATE DATABASE IF NOT EXISTS \`$DB_NAME\`;"
    docker compose exec -T db mysql -uroot -p"$DB_ROOT_PASS" "$DB_NAME" < "$TGS_RAIZ/db/schema.sql"
    echo "[....] aplicando db/seed.sql"
    docker compose exec -T db mysql -uroot -p"$DB_ROOT_PASS" "$DB_NAME" < "$TGS_RAIZ/db/seed.sql"
    echo "[OK]   esquema y usuarios cargados"
fi

echo
echo "============================================================"
echo " Instalacion completada"
echo "============================================================"
echo
echo "  Sitio web   : http://localhost:${APP_PUERTO}"
echo
echo "  Usuarios de prueba:"
echo "    admin@bluetech.com      Admin123!     (administrador)"
```

```
echo "    santiiuyy12@gmail.com   Gestor123!    (gestor)"
echo "    carlos.gomez@gmail.com  Cliente123!   (cliente)"
echo
echo "  Ver estado   : bash scripts/estado.sh"
echo "  Respaldar BD : bash scripts/respaldar.sh"
echo "  Simular caida: bash scripts/caida.sh"
echo "  Restaurar BD : bash scripts/restaurar.sh"
echo
```

5. scripts/restaurar.sh

restaurar.sh

```
#!/usr/bin/env bash
# ================================================================
# TGS - Restauracion de la base de datos desde un respaldo
#
# Uso:
#   bash scripts/restaurar.sh                 (usa el respaldo mas nuevo)
#   bash scripts/restaurar.sh tgs_20260930_120000.sql
#
# Vacia las tablas del contenedor y vuelve a cargar el .sql elegido.
# ================================================================
set -euo pipefail

source "$(dirname "${BASH_SOURCE[0]}")/_entorno.sh"

cd "$TGS_RAIZ"

echo "============================================================"
echo " TGS - Restauracion de la base de datos"
echo "============================================================"

tgs_cargar_entorno

ARCHIVO="${1:-}"

# Sin argumento: se toma el respaldo mas reciente
if [ -z "$ARCHIVO" ]; then
    ARCHIVO="$(find "$TGS_RAIZ/backups" -maxdepth 1 -name 'tgs_*.sql' -printf '%T@ %p\n' 2>/dev/null \
        | sort -rn | head -1 | cut -d' ' -f2- || true)"

    if [ -z "$ARCHIVO" ]; then
        echo "ERROR: no hay ningun respaldo en backups/ para restaurar." >&2
        echo "       Genera uno con: bash scripts/respaldar.sh" >&2
        exit 1
    fi
fi

# Permite pasar solo el nombre o la ruta completa
if [ ! -f "$ARCHIVO" ]; then
    ARCHIVO="$TGS_RAIZ/backups/$ARCHIVO"
fi

if [ ! -f "$ARCHIVO" ]; then
    echo "ERROR: no existe el respaldo '$ARCHIVO'." >&2
    echo "       Disponibles:" >&2
    ls -1 "$TGS_RAIZ/backups" 2>/dev/null | sed 's/^/         /' >&2
    exit 1
fi

if ! docker compose ps --status running --services 2>/dev/null | grep -q '^db$'; then
    echo "ERROR: el contenedor de la base de datos no esta corriendo." >&2
    echo "       Levantalo con: bash scripts/instalar.sh" >&2
    exit 1
fi

echo
echo "  Respaldo a restaurar : $(basename "$ARCHIVO")"
echo "  Tamano               : $(du -h "$ARCHIVO" | cut -f1)"
echo "  Base de datos        : $DB_NAME"
echo
echo "  ATENCION: esto reemplaza los datos actuales del contenedor."
echo

# Confirmacion explicita, para no perder trabajo por un doble clic equivocado
read -r -p "  Confirmar la restauracion? (si/NO): " RESPUESTA
if [ "$RESPUESTA" != "si" ]; then
    echo "  Restauracion cancelada."
    exit 0
fi

echo
echo "[1/3] Preparando la base de datos"

# El respaldo trae sus propios DROP TABLE, pero se limpia igual para
```

```
# garantizar que no queden tablas sueltas de una version anterior.
docker compose exec -T db mysql -uroot -p"$DB_ROOT_PASS" -e \
    "DROP DATABASE IF EXISTS \`$DB_NAME\`; CREATE DATABASE \`$DB_NAME\` CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"

echo "[2/3] Cargando el respaldo"
docker compose exec -T db mysql -uroot -p"$DB_ROOT_PASS" "$DB_NAME" < "$ARCHIVO"

echo "[3/3] Verificando"

TABLAS="$(docker compose exec -T db mysql -uroot -p"$DB_ROOT_PASS" -N -B -e \
    "SELECT COUNT(*) FROM information_schema.tables WHERE table_schema='$DB_NAME';" 2>/dev/null | tr -d '[:space:]')"

USUARIOS="$(docker compose exec -T db mysql -uroot -p"$DB_ROOT_PASS" -N -B -e \
    "SELECT COUNT(*) FROM \`$DB_NAME\`.\`usuario\`;" 2>/dev/null | tr -d '[:space:]')"

echo "[OK]   Restauracion completa"
echo "       Tablas en la base : ${TABLAS:-0}"
echo "       Usuarios cargados : ${USUARIOS:-0}"
echo
echo "  Sitio web: http://localhost:${APP_PUERTO}"
echo
```

6. scripts/_entorno.sh (dependencia)

_entorno.sh

```
#!/usr/bin/env bash
# ====================================================================
# TGS - Libreria compartida por los scripts de instalacion
#
# Este archivo se carga con "source", no se ejecuta directo.
# Su unico trabajo es poner las variables de .env y .env.local en el
# entorno, aplicando la precedencia del proyecto:
#
#     .env        (global, versionada)   se carga primero
#     .env.local  (local, no versionada)  se carga despues y pisa
#
# Asi Docker Compose ve exactamente los mismos valores que leeria
# config.php dentro del contenedor.
#
# NOTA: al usar "source", el contenido de los .env se interpreta como
# shell. Por eso los valores con espacios deben ir entre comillas.
# ====================================================================

# Raiz del proyecto (un nivel arriba de la carpeta scripts/)
TGS_RAIZ="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"

# Carga .env y luego .env.local, exportando todo al entorno.
tgs_cargar_entorno() {
    if [ ! -f "$TGS_RAIZ/.env" ]; then
        echo "ERROR: falta el archivo de variables globales $TGS_RAIZ/.env" >&2
        echo "       se genera solo con:  bash scripts/instalar.sh" >&2
        return 1
    fi

    set -a
    # shellcheck disable=SC1091
    . "$TGS_RAIZ/.env"

    if [ -f "$TGS_RAIZ/.env.local" ]; then
        # shellcheck disable=SC1091
        . "$TGS_RAIZ/.env.local"
    fi
    set +a

    export TGS_RAIZ
    return 0
}

# Crea .env a partir de .env.example si todavia no existe.
# El .env global no se versiona (lleva claves), asi que un clone nuevo
# no lo trae: se genera desde la plantilla para que instalar.sh funcione
# recien clonado, sin pasos manuales.
tgs_crear_env_global() {
    if [ -f "$TGS_RAIZ/.env" ]; then
        echo "[OK]   .env ya existe, se respeta"
        return 0
    fi

    if [ ! -f "$TGS_RAIZ/.env.example" ]; then
        echo "ERROR: no existe .env.example, no se puede generar .env" >&2
        return 1
    fi

    cp "$TGS_RAIZ/.env.example" "$TGS_RAIZ/.env"
    echo "[OK]   .env creado a partir de .env.example"
```

```
    return 0
}

# Crea .env.local a partir de .env.example si todavia no existe.
# Esto es lo que genera las VARIABLES LOCALES de la maquina.
tgs_crear_env_local() {
    if [ -f "$TGS_RAIZ/.env.local" ]; then
        echo "[OK]   .env.local ya existe, se respeta"
        return 0
    fi

    if [ ! -f "$TGS_RAIZ/.env.example" ]; then
        echo "ERROR: no existe .env.example, no se puede generar .env.local" >&2
        return 1
    fi

    cp "$TGS_RAIZ/.env.example" "$TGS_RAIZ/.env.local"

    # La contrasena de la plantilla no sirve: se genera una propia.
    local senha
    senha="$(head -c 18 /dev/urandom | base64 | tr -dc 'A-Za-z0-9' | head -c 16)"

    # sed -i con unaexpression segura, el separador | evita conflictos
    sed -i "s|^DB_PASS=.*|DB_PASS=Tgs${senha}|"        "$TGS_RAIZ/.env.local"
    sed -i "s|^DB_ROOT_PASS=.*|DB_ROOT_PASS=Root${senha}|" "$TGS_RAIZ/.env.local"

    # Los permisos se ajustan en tgs_ajustar_permisos_env_local(), que
    # tambien se ejecuta cuando el archivo ya existia.
    echo "[OK]   .env.local creado con contrasenas generadas para esta maquina"
    return 0
}

# El servidor web corre como www-data (uid 33) dentro del contenedor y
# tiene que poder LEER .env y .env.local. Un 600 o 640 deja la aplicacion
# sin credenciales y MySQL responde "Access denied".
#
# La solucion es 644: el archivo se lee desde el contenedor, pero sigue
# fuera de git gracias a la regla .env.local del .gitignore.
tgs_ajustar_permisos_env_local() {
    local objetivo

    for objetivo in "$TGS_RAIZ/.env.local" "$TGS_RAIZ/.env"; do
        if [ -f "$objetivo" ]; then
            chmod 644 "$objetivo"
        fi
    done
}

# Verifica que Docker este instalado y el demonio este encendido.
tgs_verificar_docker() {
    if ! command -v docker >/dev/null 2>&1; then
        echo "ERROR: Docker no esta instalado." >&2
        echo "       En Ubuntu: sudo apt install -y docker.io docker-compose-v2" >&2
        return 1
    fi

    if ! docker info >/dev/null 2>&1; then
        echo "ERROR: el demonio de Docker no responde." >&2
        echo "       Prueba con: sudo systemctl start docker" >&2
        return 1
    fi

    if ! docker compose version >/dev/null 2>&1; then
        echo "ERROR: falta el plugin docker compose (v2)." >&2
        echo "       En Ubuntu: sudo apt install -y docker-compose-v2" >&2
        return 1
    fi

    echo "[OK]   Docker $(docker --version | awk '{print $3}') / Compose $(docker compose version --short)"
    return 0
}

# Espera a que el contenedor de base de datos responda.
tgs_esperar_db() {
    local intentos=${1:-40}

    echo "[....] esperando a que MySQL acepte conexiones"

    for _ in $(seq 1 "$intentos"); do
        if docker compose -f "$TGS_RAIZ/docker-compose.yml" exec -T db \
            mysqladmin ping -h 127.0.0.1 -uroot -p"$DB_ROOT_PASS" --silent >/dev/null 2>&1; then
            echo "[OK]   MySQL responde"
            return 0
        fi
        sleep 2
    done

    echo "ERROR: MySQL no respondio a tiempo. Revisa: docker compose logs db" >&2
    return 1
}
```